



Universidade Federal Rural de Pernambuco
Departamento de Estatística e Informática
Disciplina de Elementos de Sistemas Computacionais
Júlia Karolyne Falcão Diniz Oliveira

“Coisas simples devem ser simples, coisas complexas devem ser possíveis” — Alan Kay

Projeto V - Arquitetura de Computadores

O objetivo do projeto 5 é construir o sistema de computador de propósito geral chamado Hack, integrando os chips previamente construídos, como a ULA, montada no projeto 2, e os registradores e sistemas de memória do projeto 3. A partir dessa integração, é possível formar um computador completo, capaz de executar programas escritos em linguagem de máquina.

Para compreender o funcionamento e a importância dessa construção, é essencial entender como surgiram os princípios que definem a arquitetura dos computadores modernos. A história da computação evoluiu de máquinas criadas para realizar tarefas específicas para sistemas programáveis e versáteis, baseados em uma única estrutura de hardware.

Um dos maiores avanços que possibilitou essa transformação foi o conceito de programa armazenado ou **Stored Program Concept**, onde a ideia é que programas e dados podem coexistir na mesma memória. Com esse conceito, um mesmo hardware passou a executar diversas tarefas apenas modificando o programa carregado na memória.

Em vez de construir uma máquina física diferente para cada função, como acontecia nas primeiras décadas da computação, surge esse modelo de hardware fixo e software variável, ou seja, um conjunto de instruções define o comportamento da máquina, como foi observado no projeto 4.

Desta forma, um mesmo computador pode rodar um jogo, processar textos ou realizar cálculos complexos, bastando mudar o software em execução. Esse princípio, que unifica dados e instruções em um mesmo espaço de memória, é o que torna o computador uma máquina universal.

Arquitetura de von Neumann

O modelo de von Neumann descreve como os computadores armazenam e processam informações. Ele é composto por quatro blocos principais:

- **CPU (Unidade Central de Processamento):** executa as instruções, faz a computação e controla o fluxo do programa;
- **Memória:** armazena tanto os dados quanto as instruções;
- **Dispositivos de Entrada (keyboard):** permitem que o usuário envie informações para o computador;
- **Dispositivos de Saída (screen):** exibem o resultado das operações.

O ponto principal dessa arquitetura é o ciclo buscar, decodificar e executar.

1. A CPU busca a próxima instrução na memória.
2. Decodifica o comando para entender o que deve ser feito.
3. Executa a operação, armazenando o resultado e passando à próxima instrução.

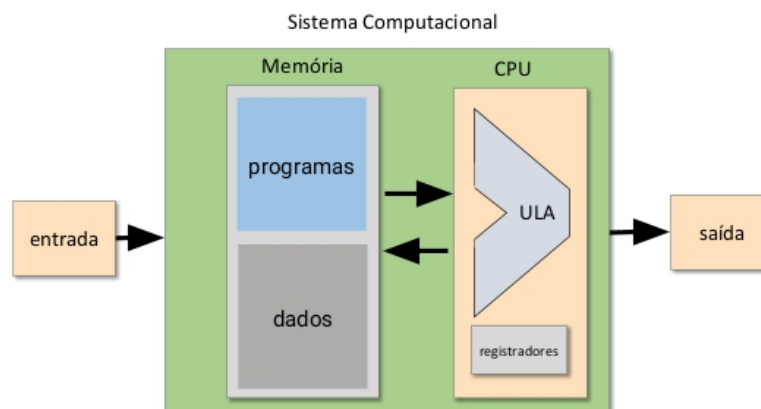


Figura 1 – Arquitetura von Neumann.
Fonte: Adaptado de Nisan; Schocken.

Nessa arquitetura conceitual, os dados e as instruções são frequentemente gerenciados dentro da mesma unidade de memória física e geralmente utilizam um único espaço de endereço.

Como resultado, não é possível alimentar simultaneamente o endereço da instrução e o endereço dos dados na única entrada de endereço, exigindo uma lógica de execução baseada em dois ciclos (fetch e execute).

Além dessa arquitetura, existe a **variante Harvard**. Nela a memória de dados e a memória de instruções são mantidas em unidades de **memória física separadas**, cada uma com espaços de endereço distintos, como na Figura 2.

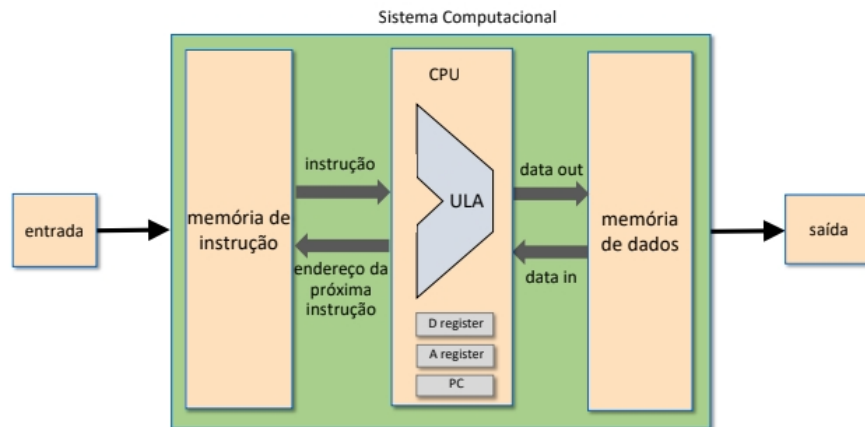


Figura 2 – Arquitetura utilizada .
 Fonte: Adaptado de Nisan; Schocken.

Esta é a configuração usada pelo computador Hack. Essa separação permite que o Hack utilize uma lógica de execução de ciclo único (single-cycle fetch-execute logic), embora o preço desse modelo de hardware mais simples seja que os programas não podem ser alterados dinamicamente.

O computador Hack é formado por:

- Uma CPU (com registradores A, D e PC);
- Uma memória de dados (RAM16K, tela e teclado mapeados na memória);
- Uma memória de instruções (ROM32K);
- E dispositivos de entrada/saída (keyboard/screen) controlados por endereços de memória específicos (memória mapeada).

Interligando os Projetos

Os capítulos e projetos anteriores forneceram todas as peças fundamentais que agora serão integradas para formar o computador Hack completo. Cada um teve um papel essencial na construção gradual da arquitetura, permitindo compreender como circuitos simples podem evoluir até se tornarem um sistema capaz de executar programas complexos.

Por isso, antes de iniciar a construção do Hack, precisamos relembrar alguns conceitos, para compreender como interligar todas as partes e a sua função no sistema Hack.

Projeto 1 - Lógica Booleana

O primeiro projeto introduziu os blocos fundamentais do hardware digital, construídos a partir de portas lógicas. Entre essas portas, vale lembrar que:

- **Mux16:** permite selecionar entre dois sinais de 16 bits, funcionando como um “interruptor” controlado por um bit seletor. Esse componente é amplamente utilizado em toda a arquitetura Hack para decidir quais dados serão enviados ou armazenados a cada momento;
- **DMux:** divide um sinal de entrada em duas saídas possíveis, dependendo do seletor, sendo útil para encaminhar dados a diferentes destinos.

Além desses chips, And, Not e Or também poderão ser usados para estruturar a construção do computador Hack, contudo os conceitos Mux e DMux são interessantes de lembrar devido ao seu nível maior de complexidade.

Projeto 2 - Lógica Aritmética

No segundo projeto, foram construídos os somadores, como também a ALU (Arithmetic Logic Unit), parte essencial da CPU.

A ALU é responsável por executar operações aritméticas e lógicas fundamentais, como soma, subtração, negação e comparações.

Ela recebe dois valores de entrada (x e y) e, a partir de seis bits de controle, define qual operação deve ser realizada. O resultado é fornecido na saída out, junto com dois sinais importantes:

- **zr (zero):** indica se o resultado foi zero;
- **ng (negative):** indica se o resultado é negativo.

Esses sinais são essenciais para os comandos de salto (*jump*) da CPU, pois determinam as condições lógicas que controlam o fluxo de execução dos programas.

Projeto 3 - Memórias

O terceiro projeto tratou dos dispositivos de armazenamento, essenciais para que o computador “lembre” valores entre uma operação e outra. Os dois Chips construídos essenciais são o Register e o RAM16K, onde

- **Register:** armazena uma palavra (16 bits). Ele é usado para guardar valores temporários dentro da CPU, como os registradores A e D;
- **RAM16K:** é a principal memória de dados do computador Hack.

Além disso, é importante saber que a hierarquia de memórias (cada nível de RAM construído combinando unidades menores) permitiu compreender como o computador acessa endereços específicos de dados e realiza operações de leitura e escrita controladas pelo sinal **load**.

No Projeto 5, essa memória será integrada ao chip Memory, junto com o mapeamento de entrada (keyboard) e saída (screen), formando o **espaço completo de endereçamento de dados**.

Projeto 4 - Linguagem de Máquina

No quarto projeto, o foco foi no nível de instruções, isto é, como os programas comunicam-se com o hardware.

A linguagem de máquina Hack define o formato das instruções que a CPU deve interpretar, divididas em dois tipos:

- **A-instruction (@value):** carrega um valor constante ou endereço no registrador A;
- **C-instruction (dest = comp; jump):** executa uma operação através da ALU e decide onde armazenar o resultado e se deve haver salto na execução.

Essa camada de software estabelece o **conjunto de instruções do computador Hack**.

A compreensão dessa linguagem é essencial para construir o chip CPU, pois ele deverá decodificar e executar exatamente essas instruções no nível de hardware.

Após essa revisão do que já fizemos, agora vamos compreender o funcionamento de cada chip, iniciando pela implementação da CPU, depois Memória e, por último, ligando os dois para formar o computador Hack.

Chip CPU

A CPU (Central Processing Unit) é o núcleo de processamento do computador Hack. Ela executa instruções, conduz operações aritméticas e lógicas por meio da ULA e controla o fluxo de execução, articulando hardware e software ao traduzir instruções de máquina em operações no circuito.

Esse chip articula três subsistemas essenciais: a ULA, os registradores internos e a lógica de controle. A ULA executa as operações indicadas em **comp**, os registradores armazenam valores e endereços temporários, e a lógica de controle interpreta o formato da instrução para definir o próximo passo da execução. O ciclo operacional consiste em buscar a instrução na ROM, decodificá-la, realizar a operação correspondente, atualizar os registradores e escolher a instrução seguinte, por continuidade ou salto.

Componentes Internos

A estrutura da CPU Hack é formada por:

- **Registrador A:** que recebe constantes de A-instructions ou endereços para acesso à RAM e serve como entrada da ULA;
- **Registrador D:** destinado ao armazenamento de valores intermediários;
- **PC (Program Counter):** responsável por manter o endereço da próxima instrução e permitir desvios condicionais;
- **ULA:** que executa as operações definidas pelo campo comp, produzindo o resultado e os sinais zr e ng.

Esses elementos são coordenados pela lógica de controle, que seleciona as rotas de dados por meio de multiplexadores e aplica os sinais derivados das C-instructions para conduzir o fluxo de execução.

Interpretação das Instruções

A CPU interpreta dois tipos de instruções:

- **A-instruction (@value)**
 - Carrega um valor constante no registrador A.
 - Exemplo: @5: o registrador A receberá o valor 5.
- **C-instruction (dest = comp ; jump)**
 - Define a operação que a ULA deve realizar (comp), onde armazenar o resultado (dest), e se haverá salto na execução (jump).

A lógica de controle decide as entradas da ULA, os registradores que devem ser escritos e se o PC será modificado conforme zr e ng.

Fluxo de Funcionamento

O processo do chip CPU consiste em

1. A CPU recebe uma instrução de 16 bits (instruction).
2. Se for uma A-instruction, o valor dos 15 bits menos significativos é armazenado em A.
3. Se for uma C-instruction, os bits são divididos em três grupos:
 - a. **comp:** define a operação da ULA.
 - b. **dest:** indica o destino do resultado (A, D ou M).
 - c. **jump:** define o comportamento do PC (se haverá salto).
4. O resultado da ULA (outM) pode ser armazenado:
 - a. No registrador D,
 - b. No registrador A,
 - c. Ou enviado para a memória RAM (caso dest = M).
5. O PC é incrementado, a menos que o campo jump e os sinais zr/ng indiquem um desvio condicional.

A Figura 3 ilustra como os sinais circulam.

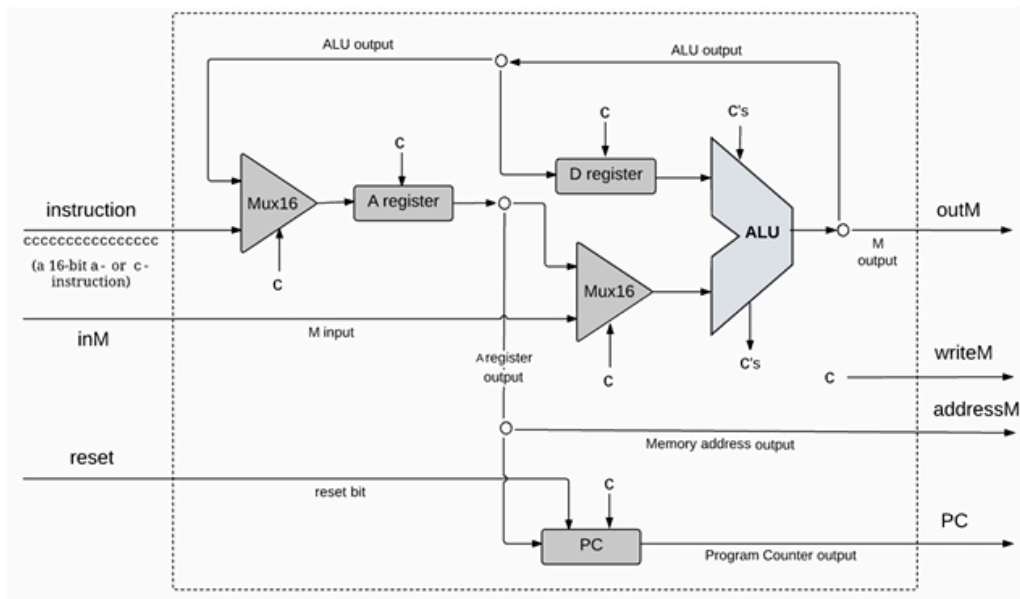


Figura 3 – Diagrama do Chip CPU.
Fonte: Adaptado de Nisan; Schocken.

Assim, a CPU é o núcleo inteligente do computador Hack, coordenando a comunicação entre a memória e a ULA, e tornando possível a execução de programas de forma autônoma e contínua.

Dica de Implementação: Use os chips Not, Mux16, Or, ARegister, And, ALU, DRegister e PC.

Chip Memory

O chip Memory é a memória do Hack, que reúne a memória de dados e os dispositivos de entrada/saída mapeados em memória (tela e teclado). Ele fornece ao sistema um espaço de endereçamento unificado que a CPU usa para ler e escrever dados. O Memory é composto por:

- **RAM16K:** armazenamento principal;
- **Screen:** memória dedicada à tela, onde cada bit representa um pixel;
- **Keyboard:** um registrador que espelha o estado do teclado (0 = nenhuma tecla pressionada).

A seleção entre esses subcomponentes é feita por decodificação de faixa: o valor de **address** determina qual deles será ativado.

Organização de endereçamento

O espaço de endereçamento do Memory é dividido assim:

Parte	Faixa de endereços	Quantidade de Palavras
RAM (dados)	0 a 16383	16K
Screen (memória de saída)	16384 a 24575	8K
Keyboard (mapa do teclado)	24576	1

O Memory faz a lógica de decodificação de faixa (address decoding), na qual a parte alta dos bits de address seleciona qual desses sub-dispositivos será acessado, e os bits de menor ordem indexam a posição interna.

Essas faixas permitem que a CPU use um único barramento de endereços para acessar dados comuns, pixels da tela e o estado do teclado.

Relembrando o Mapeamento da Tela SCREEN

A tela física do Hack tem 256 linhas × 512 colunas. Quando a CPU escreve uma palavra no endereço correspondente, os 16 pixels daquela “palavra” da tela são atualizados. Relembre a relação de palavra, bits e pixels observando a Figura 4.

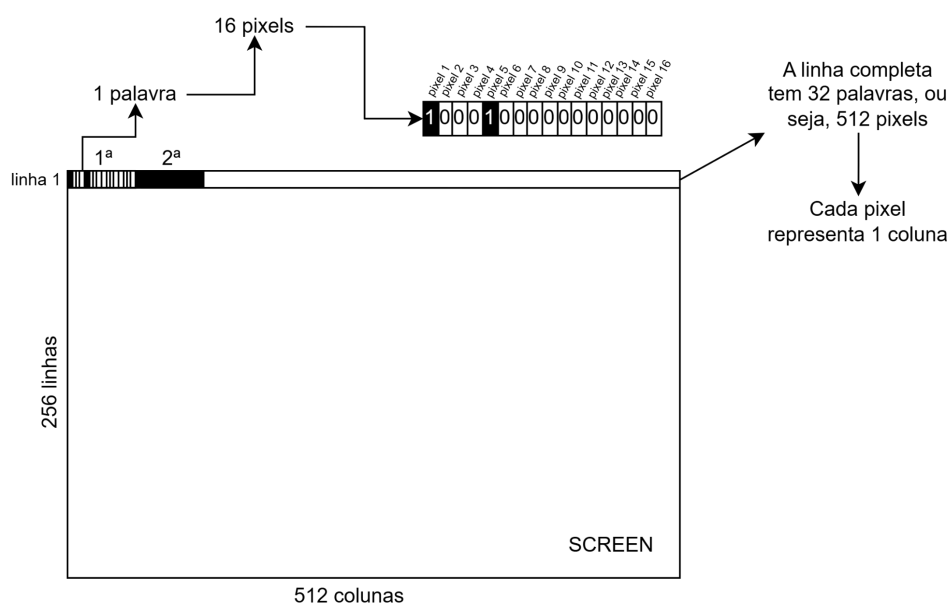


Figura 4 – Representação ilustrativa da tela.
Fonte: Autora.

Fluxo de Funcionamento

O processo do chip Memory consiste em:

1. Decodificar o endereço para escolher entre RAM, Screen ou Keyboard;
2. Direcionar os sinais **in** e **load** apenas ao subcomponente selecionado;
3. Multiplexar a saída (out) entre RAM16K, Screen e Keyboard;

4. Usar apenas os bits necessários para indexação interna (14 bits para RAM16K, 13 bits para Screen, nenhum para Keyboard).

Essa estrutura permite que toda a memória lógica do computador (dados, dispositivo de entrada e dispositivo de saída) seja acessada de forma uniforme através de um único barramento.

Dica de Implementação: Use os Chips Mux16, RAM16K, Screen, Keyboard e DMux.

Chip Computer

O chip Computer é o topo da hierarquia de hardware do Hack. Ele integra a CPU, a memória de instruções (ROM32K) e a memória de dados (Memory) em um único sistema que pode executar programas completos. Em outras palavras, é o “computador” propriamente dito, o circuito final que, ao receber reset = 0, começa a executar o programa armazenado na ROM.

Fluxo de Funcionamento

O processo do chip Computer é composto pelas etapas:

1. Alimente a ROM32K com o endereço do próximo comando (vindo do pc da CPU);
2. Forneça à CPU a instrução atual (saída da ROM) e o valor lido da memória de dados (inM);
3. Receba de volta da CPU os sinais necessários para leitura/escrita na memória de dados (addressM, outM, writeM);
4. Coordene o reset para iniciar/reiniciar a execução do programa.

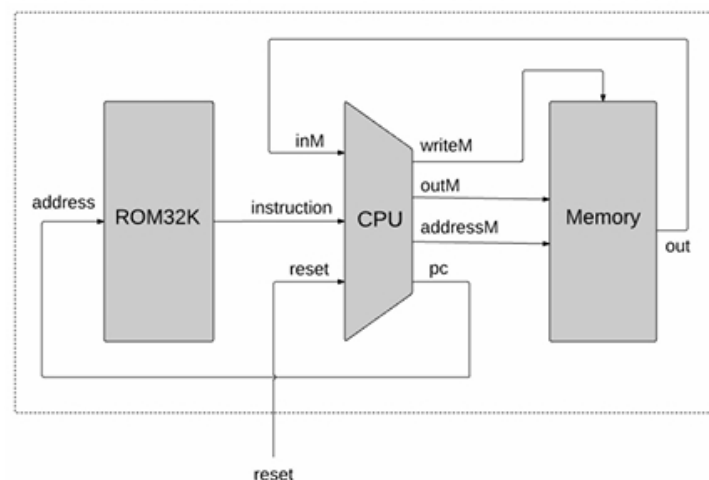


Figura 5 – Diagrama do Chip Computer.
Fonte: Adaptado de Nisan; Schocken.

Dica de Implementação: Use os chips ROM32K, CPU e Memory.

Comparação Hack e RISC/CISC

Ao concluir o Projeto 5, o computador Hack representa o ponto em que todos os conceitos de hardware se unem, formando uma arquitetura funcional e universal.

Apesar de ser uma máquina simplificada, ele segue princípios semelhantes aos das arquiteturas RISC (Reduced Instruction Set Computer) e CISC (Complex Instruction Set Computer).

Compreender essa comparação ajuda a conectar o Hack com o funcionamento dos processadores reais.

Arquitetura RISC

A filosofia RISC foi criada para tornar o hardware mais simples e rápido, priorizando instruções curtas e uniformes que possam ser executadas em um único ciclo de clock.

O foco está na eficiência de hardware e velocidade de execução, em vez da complexidade das instruções.

Características principais:

- Conjunto reduzido de instruções, geralmente todas com o mesmo formato e tamanho fixo.
- Execução em ciclo único (uma instrução por ciclo).
- Controle simples e direto, onde reduz a lógica de decodificação.
- Operações realizadas entre registradores, minimizando acessos à memória.
- Pipeline facilitado, pois as instruções são previsíveis e uniformes.
- Exemplos de processadores RISC: MIPS, ARM, RISC-V, SPARC.

Arquitetura CISC

Já a filosofia CISC (Complex Instruction Set Computer) adota o caminho oposto: possui um conjunto extenso e variado de instruções, algumas muito complexas, capazes de realizar tarefas completas com uma única operação.

O objetivo é diminuir o número de instruções por programa, mesmo que cada instrução seja mais demorada.

Características principais:

- Conjunto grande e variado de instruções, incluindo operações de alto nível.
- Ciclos de execução múltiplos, ou seja, nem todas as instruções levam o mesmo tempo.

- Decodificação mais complexa, exigindo mais transistores e controle interno.
- Integração entre operações de memória e cálculo (muitos operandos vêm diretamente da memória).
- Ideal para compatibilidade retroativa, como nas famílias Intel x86.

Onde o Hack se encaixa

O computador Hack segue nitidamente a filosofia RISC, tanto na estrutura quanto no comportamento. Assim, o Hack é uma representação minimalista de um processador RISC, com apenas dois tipos de instruções (A e C), execução em ciclo único, e um conjunto muito limitado de registradores (A, D e PC).

Sua simplicidade torna possível visualizar cada etapa do processamento, compreender o fluxo de dados e observar como hardware e software interagem de forma direta.

Enquanto processadores reais adicionam camadas de otimização (pipeline, cache, branch prediction), o Hack mantém o essencial, sendo os mesmos princípios que sustentam qualquer arquitetura moderna.

Conclusão

Estudar e implementar o computador Hack permite entender na prática os conceitos fundamentais que se mantêm nas arquiteturas reais. Ele demonstra que, mesmo com poucos componentes, como ULA, registradores, memória e controle de instruções, é possível formar um sistema completo, funcional e programável.

Assim, o Hack não é apenas um modelo acadêmico, mas também uma tradução concreta da filosofia RISC, mostrando como a simplicidade e a clareza de projeto são a base sobre a qual toda a computação moderna foi construída.

Dicas Finais

- Não pense que está atribuindo valores, mas sim trabalhando com fios e cada pedaço dele, ao passar por uma porta lógica, recebe um nome simbólico;
- Tente quebrar cada problema em partes menores antes de interligar as portas necessárias para construir o chip;
- Recomendo testar na própria [web-ide do Nand2Tetris](#);
- Rabisque um diagrama para representar a porta lógica antes de implementar.

Referências

NISAN, Noam; SCHOCKEN, Shimon. *The elements of computing systems: building a modern computer from first principles*. Cambridge: MIT Press, 2005.